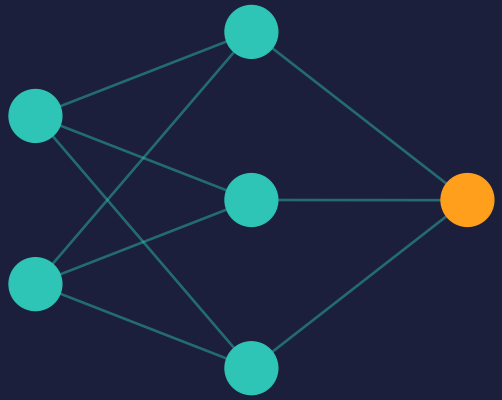
دوره یادگیری عمیق و شبکه‌های عصبی



جلسه ۴

حلقه آموزش استاندارد: nn.Module، DataLoader و Fashion-MNIST

ساختار یک پروژه کامل PyTorch از داده تا ذخیره مدل

کیوان محمودی

در این جلسه

۱ دیتاست Fashion-MNIST

۲ torchvision و transforms

۳ Train / Validation / Test

۴ Dataset و DataLoader

۵ تعریف مدل با nn.Module

۶ Logits و CrossEntropyLoss

۷ توابع train_one_epoch و evaluate

۸ تحلیل نتایج، ذخیره و پیش‌بینی

دیتاست Fashion-MNIST



- ۷۰,۰۰۰ تصویر خاکستری ۲۸×۲۸
- ۶۰,۰۰۰ آموزشی و ۱۰,۰۰۰ آزمون
- ۱۰ کلاس لباس و کفش
- جایگزین سخت‌تر MNIST (Xiao et al., 2017)

مسیر داده در PyTorch



Dataset نمونه‌ها را تولید می‌کند، transforms آن‌ها را آماده می‌کند و DataLoader دسته‌ها را به مدل می‌دهد

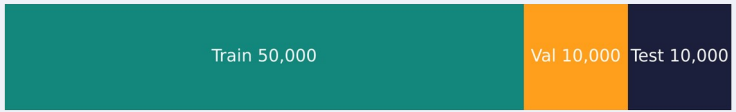
بارگذاری و نرمال‌سازی داده

```
transform = transforms.Compose([
    transforms.ToTensor(),                # [0,255] -> [0,1], (C,H,W)
    transforms.Normalize((MEAN,), (STD,)), # stats from TRAIN only
])
full_train = datasets.FashionMNIST('data', train=True,
                                   download=True, transform=transform)
test_set = datasets.FashionMNIST('data', train=False,
                                 download=True, transform=transform)
train_set, val_set = random_split(full_train, [50_000, 10_000])
```

- ToTensor تصویر را به float32 در بازه صفر تا یک می‌برد.
- Normalize میانگین را صفر و انحراف معیار را یک می‌کند.
- آمار نرمال‌سازی فقط از داده آموزشی، برای جلوگیری از Data Leakage.

سه بخش داده و نقش هر کدام

- Train: به‌روزرسانی وزن‌ها
- Validation: انتخاب ابرپارامتر و پایش Overfitting
- Test: فقط یک بار و برای گزارش نهایی
- استفاده از Test برای تصمیم‌گیری، نتیجه را خوش‌بینانه می‌کند.



Dataset و DataLoader

```
class TensorPairDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

train_loader = DataLoader(train_set, batch_size=128,
                          shuffle=True, num_workers=2)
X, y = next(iter(train_loader))
# X: (128, 1, 28, 28) float32   y: (128,) int64
```

- Dataset فقط دو متد لازم دارد: __len__ و __getitem__
- DataLoader دسته‌بندی، Shuffle و بارگذاری موازی را انجام می‌دهد.
- داده آموزشی Shuffle می‌شود؛ داده ارزیابی نه.

تعریف مدل با nn.Module

```
class MLP(nn.Module):
    def __init__(self, hidden=(256, 128), n_classes=10):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28 * 28, hidden[0])
        self.fc2 = nn.Linear(hidden[0], hidden[1])
        self.out = nn.Linear(hidden[1], n_classes)
        self.act = nn.ReLU()

    def forward(self, x):
        x = self.flatten(x)
        x = self.act(self.fc1(x))
        x = self.act(self.fc2(x))
        return self.out(x)        # logits, no softmax

model = MLP().to(device)        # 235,146 parameters
```

- لایه‌ها در __init__، مسیر داده در forward.
- همیشه model(x) را صدا بزنید، نه forward را.
- برای مدل‌های ساده ترتیبی: nn.Sequential

Logits و CrossEntropyLoss

قاعده

- لایه آخر مدل Softmax ندارد؛ خروجی Logits است.
- nn.CrossEntropyLoss خودش Softmax و Log را پایدار ترکیب می‌کند.
- برچسب‌ها اعداد صحیح int64 هستند، نه One-hot.

بررسی سلامت (Sanity Check)

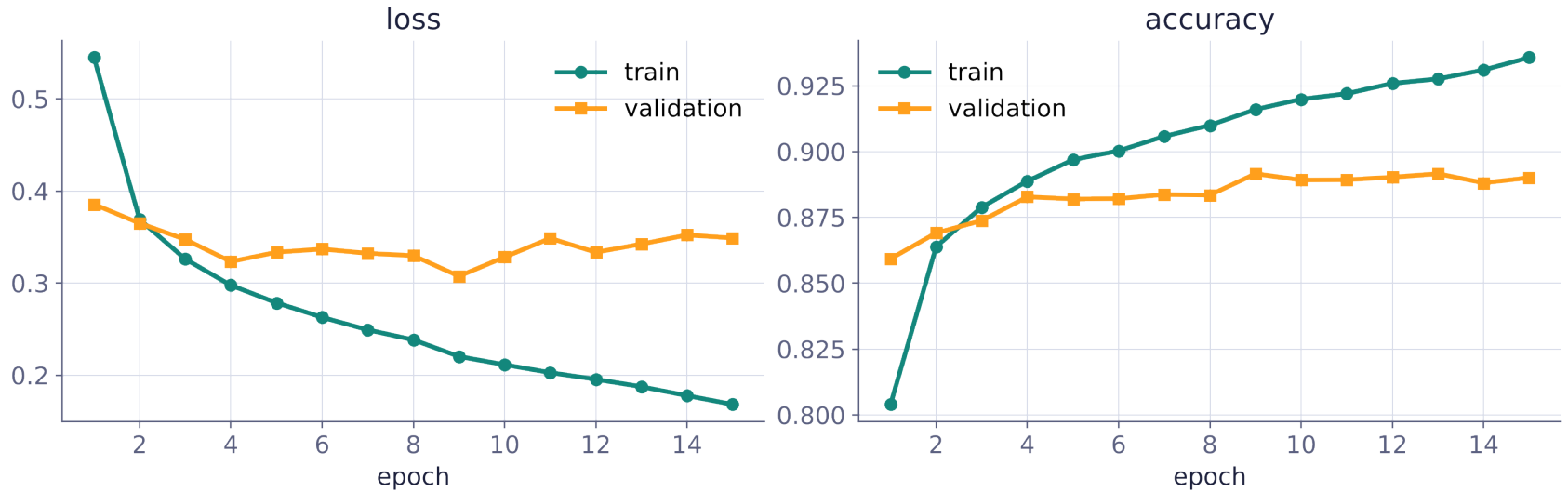
- مدل آموزش‌ندیده به ۱۰ کلاس احتمال تقریباً برابر می‌دهد.
- Loss اولیه باید نزدیک $\log 10 \approx 2.30$ باشد.
- Loss اولیه ما: ۲/۳۲۵
- اگر خیلی متفاوت بود، قبل از آموزش اشکال را پیدا کنید.

تابع آموزش یک Epoch

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss, correct, n = 0.0, 0, 0
    for X, y in loader:
        X, y = X.to(device), y.to(device)
        logits = model(X)
        loss = criterion(logits, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(y)
        correct += (logits.argmax(1) == y).sum().item()
        n += len(y)
    return total_loss / n, correct / n
```

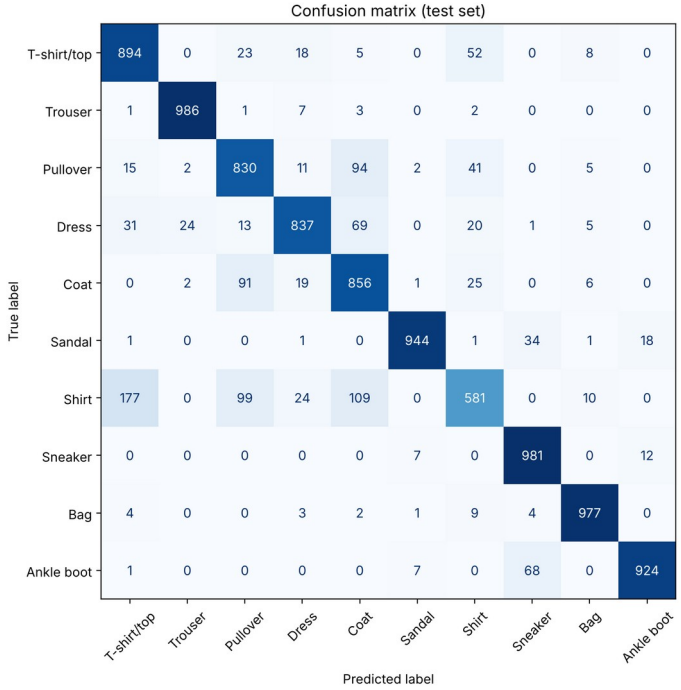
- model.train در ابتدای آموزش.
- داده و مدل روی یک device.
- تابع evaluate مشابه است، با model.eval و torch.no_grad
- این دو تابع تا پایان دوره استفاده می‌شوند.

منحنی‌های آموزش: SGD + Momentum با Epoch ۱۵



Loss آموزش پیوسته کم می‌شود ولی Loss اعتبارسنجی از Epoch نهم بالا می‌رود: نشانه Overfitting

Confusion Matrix: مدل کجا اشتباه می‌کند؟



- دقت روی داده آزمون: ۸۸/۱٪
- بیشترین خطا: Shirt با T-shirt، Pullover و Coat
- MLP تصویر را به بردار ۷۸۴تایی تبدیل می‌کند و ساختار فضایی را نادیده می‌گیرد.
- راه‌حل: شبکه کانولوشنی (جلسه ۷)

ذخیره مدل و پیش‌بینی (Inference)

ذخیره و بارگذاری

- روش توصیه‌شده: ذخیره state_dict
- torch.save(model.state_dict, path)
- ابتدا نمونه‌ای از همان کلاس، سپس load_state_dict
- map_location برای بارگذاری روی CPU/GPU دیگر

پیش‌بینی روی یک نمونه

- model.eval و torch.no_grad
- افزودن بعد Batch با unsqueeze(0) به ورودی
- softmax روی Logits برای دیدن احتمال‌ها
- argmax برای کلاس نهایی

جمع‌بندی

- مسیر استاندارد داده: Dataset → transforms → DataLoader → model
- Train / Validation / Test؛ داده آزمون فقط برای گزارش نهایی
- مدل = nn.Module با __init__ و forward؛ خروجی Logits
- CrossEntropyLoss خودش Softmax را دارد
- model.train و model.eval را هرگز فراموش نکنید
- MLP به ۸۸٪ می‌رسد؛ برای تصویر به CNN نیاز داریم

تمرین

۱ ابعاد لایه‌های پنهان را به (512, 256) و (64, 32) تغییر دهید و اثر آن را روی Overfitting گزارش کنید.

۲ Normalize را حذف کنید و سرعت همگرایی را مقایسه کنید.

۳ یک Dataset سفارشی برای Churn_Modelling.csv بنویسید و یک MLP آموزش دهید.

۴ بهترین مدل (بیشترین دقت اعتبارسنجی) را حین آموزش ذخیره کنید و همان را روی Test بسنجید.